Heart Disease Classification Pipeline with funcml

From Formula Definition to Exported Results

Imad El Badisy

2026-05-20

Table of contents


```
library(funcml)

results_dir <- "heartdisease_pipeline_results"
dir.create(results_dir, showWarnings = FALSE, recursive = TRUE)

set.seed(42)
```

1 Objective

This document builds a complete binary classification pipeline using the internal `heartdisease` dataset shipped with **funcml**. The target is `heart_disease` (`"No"` / `"Yes"`), and the workflow proceeds from an explicit model formula to model fitting, resampling-based evaluation, tuning, comparison, interpretation, calibration, prediction, and exported results.

2 Data

2.1 Load and Prepare

```
data_heart <- heartdisease

data_heart$sex <- factor(data_heart$sex)
data_heart$chest_pain <- factor(data_heart$chest_pain)
data_heart$blood_sugar <- factor(
  data_heart$blood_sugar,
  levels = c(FALSE, TRUE),
  labels = c("normal", "high")
)
data_heart$exercise_induced_angina <- factor(data_heart$exercise_induced_angina)
data_heart$heart_disease <- factor(
  data_heart$heart_disease,
  levels = c("No", "Yes")
)

data_cls <- data_heart
str(data_cls)
```

```
'data.frame':   303 obs. of  9 variables:
 $ age                   : int  63 67 67 37 41 56 62 57 63 53 ...
 $ sex                   : Factor w/ 2 levels "Female","Male": 2 2 2 2 1 2 1 1 2 2 ...
 $ chest_pain            : Factor w/ 4 levels "Asymptomatic",..: 4 1 1 3 2 2 1 1 1 1 ...
 $ bp                    : int  145 160 120 130 130 120 140 120 130 140 ...
 $ cholesterol           : int  233 286 229 250 204 236 268 354 254 203 ...
 $ blood_sugar           : Factor w/ 2 levels "normal","high": 2 1 1 1 1 1 1 1 1 2 ...
 $ maximum_hr            : int  150 108 129 187 172 178 160 163 147 155 ...
 $ exercise_induced_angina: Factor w/ 2 levels "No","Yes": 1 2 2 1 1 1 1 2 1 2 ...
 $ heart_disease         : Factor w/ 2 levels "No","Yes": 1 2 2 1 1 1 2 1 2 2 ...
```

```
table(data_cls$heart_disease)
```

```
 No Yes
164 139
```

2.2 Explicit Model Formula

```
formula_cls <- heart_disease ~ age + sex + chest_pain + bp + cholesterol +
  blood_sugar + maximum_hr + exercise_induced_angina

formula_cls
```

```
heart_disease ~ age + sex + chest_pain + bp + cholesterol + blood_sugar +
    maximum_hr + exercise_induced_angina
```

3 Baseline Model

We start with a random forest classifier because it is a strong tabular baseline and supports probability prediction.

```
fit_baseline <- fit(
  formula = formula_cls,
  data    = data_cls,
  model   = "ranger",
  spec    = list(num.trees = 300, mtry = 3, seed = 42)
)

fit_baseline
```

```
<funcml_fit> classification model: ranger
Formula: heart_disease ~ age + sex + chest_pain + bp + cholesterol + blood_sugar +
 Formula:     maximum_hr + exercise_induced_angina
Features: 10 | Obs: 303
```

```
pred_class <- predict(fit_baseline, newdata = data_cls)
pred_prob  <- predict(fit_baseline, newdata = data_cls, type = "prob")

predictions <- data.frame(
  id = seq_len(nrow(data_cls)),
  truth = data_cls$heart_disease,
  predicted_class = pred_class,
  prob_no = pred_prob[, "No"],
  prob_yes = pred_prob[, "Yes"]
)

head(predictions)
```

```
  id truth predicted_class    prob_no   prob_yes
1  1    No              No 0.75640176 0.24359824
2  2   Yes             Yes 0.06032544 0.93967456
3  3   Yes             Yes 0.06363946 0.93636054
4  4    No              No 0.96473532 0.03526468
5  5    No              No 0.98983219 0.01016781
6  6    No              No 0.92590315 0.07409685
```

4 Resampling-Based Evaluation

```
eval_baseline <- evaluate(
  data       = data_cls,
  formula    = formula_cls,
  model      = "ranger",
  resampling = cv(v = 5, seed = 42),
  metrics    = c("accuracy", "auc", "logloss"),
  type       = "prob",
  seed       = 42
)

eval_baseline
```

```
<funcml_eval> model: ranger | task: classification
    metric      mean         sd n  std_error conf_level  conf_low conf_high
1 accuracy 0.7427063 0.05303319 5 0.02371716       0.95 0.6768569 0.8085557
2      auc 0.8314875 0.05263127 5 0.02353742       0.95 0.7661371 0.8968379
3  logloss 0.5086376 0.05794494 5 0.02591376       0.95 0.4366895 0.5805858
```

```
eval_baseline$summary
```

```
    metric      mean         sd n  std_error conf_level  conf_low conf_high
1 accuracy 0.7427063 0.05303319 5 0.02371716       0.95 0.6768569 0.8085557
2      auc 0.8314875 0.05263127 5 0.02353742       0.95 0.7661371 0.8968379
3  logloss 0.5086376 0.05794494 5 0.02591376       0.95 0.4366895 0.5805858
```

```
plot(eval_baseline)
```

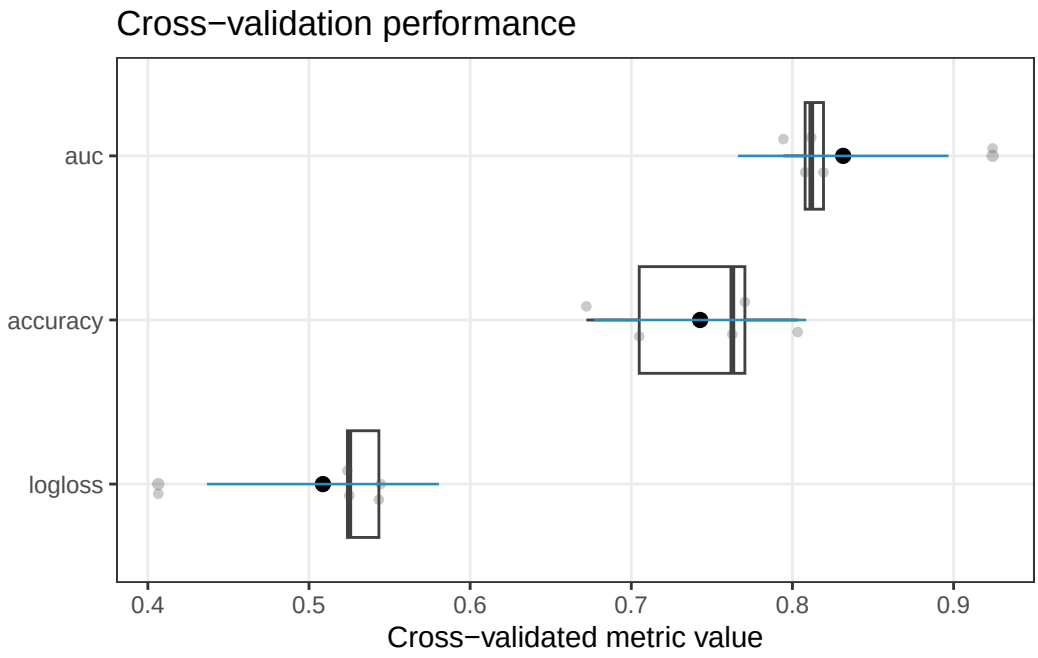

Figure 1: Cross-validated performance for the baseline random forest classifier.

5 Hyperparameter Tuning

```
grid_ranger <- expand.grid(
  num.trees = c(100, 200, 300),
  mtry      = c(2, 3, 5)
)

tuned_ranger <- tune(
  data       = data_cls,
  formula    = formula_cls,
  model      = "ranger",
  grid       = grid_ranger,
  resampling = cv(v = 5, seed = 42),
  metric     = "auc",
  type       = "prob",
  search     = "grid",
  seed       = 42
)

tuned_ranger$best
```

```
  num.trees mtry      mean         sd n  std_error conf_level  conf_low
2       200    2 0.8380562 0.05005723 5 0.02238627       0.95 0.7759019
  conf_high
```

```
2 0.9002104
```

```
plot(tuned_ranger)
```

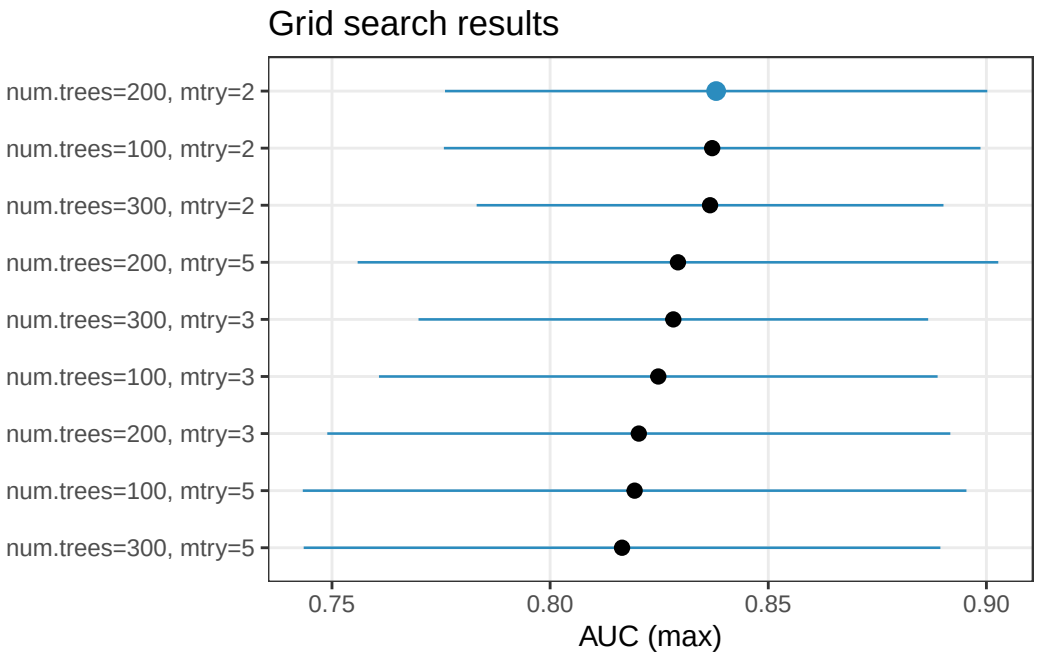

Figure 2: Ranger tuning results ranked by cross-validated AUC.

6 Final Model

```
fit_final <- fit(
  formula = formula_cls,
  data    = data_cls,
  model   = "ranger",
  spec    = as.list(tuned_ranger$best[1, c("num.trees", "mtry")])
)

fit_final
```

```
<funcml_fit> classification model: ranger
Formula: heart_disease ~ age + sex + chest_pain + bp + cholesterol + blood_sugar +
 Formula:     maximum_hr + exercise_induced_angina
Features: 10 | Obs: 303
```

```
final_class <- predict(fit_final, newdata = data_cls)
final_prob  <- predict(fit_final, newdata = data_cls, type = "prob")
```

```r
final_predictions <- data.frame(
  id = seq_len(nrow(data_cls)),
  truth = data_cls$heart_disease,
  predicted_class = final_class,
  prob_no = final_prob[, "No"],
  prob_yes = final_prob[, "Yes"]
)

head(final_predictions)
```

```
  id truth predicted_class    prob_no   prob_yes
1  1    No              No 0.72353386 0.27646614
2  2   Yes             Yes 0.07443297 0.92556703
3  3   Yes             Yes 0.10900334 0.89099666
4  4    No              No 0.93150942 0.06849058
5  5    No              No 0.97224799 0.02775201
6  6    No              No 0.83529266 0.16470734
```

7 Learner Comparison

```r
cmp_cls <- compare_learners(
  data       = data_cls,
  formula    = formula_cls,
  models     = c("glm", "rpart", "ranger"),
  resampling = cv(v = 5, seed = 42),
  metrics    = c("accuracy", "auc", "logloss"),
  type       = "prob",
  seed       = 42
)

cmp_cls
```

```
<funcml_compare> task: classification | tuned: FALSE
   model   metric      mean         sd n  std_error conf_level  conf_low
1    glm accuracy 0.7760489 0.04670526 5 0.02088723       0.95 0.7180567
2    glm      auc 0.8570887 0.03980879 5 0.01780303       0.95 0.8076596
3    glm  logloss 0.4862753 0.06847182 5 0.03062153       0.95 0.4012564
4  rpart accuracy 0.7425952 0.06285163 5 0.02810810       0.95 0.6645546
5  rpart      auc 0.8045154 0.05262589 5 0.02353501       0.95 0.7391717
6  rpart  logloss 0.6492840 0.15445732 5 0.06907541       0.95 0.4574999
7 ranger accuracy 0.7559322 0.04442057 5 0.01986548       0.95 0.7007768
8 ranger      auc 0.8299723 0.05334777 5 0.02385785       0.95 0.7637323
9 ranger  logloss 0.5055037 0.05966646 5 0.02668365       0.95 0.4314180
  conf_high tuned rank
```

```
1 0.8340411 FALSE    1
2 0.9065179 FALSE    1
3 0.5712943 FALSE    1
4 0.8206358 FALSE    3
5 0.8698591 FALSE    3
6 0.8410681 FALSE    3
7 0.8110876 FALSE    2
8 0.8962124 FALSE    2
9 0.5795894 FALSE    2
```

```
cmp_cls$results
```

```
   model   metric      mean         sd n  std_error conf_level  conf_low
1    glm accuracy 0.7760489 0.04670526 5 0.02088723       0.95 0.7180567
2    glm      auc 0.8570887 0.03980879 5 0.01780303       0.95 0.8076596
3    glm  logloss 0.4862753 0.06847182 5 0.03062153       0.95 0.4012564
4  rpart accuracy 0.7425952 0.06285163 5 0.02810810       0.95 0.6645546
5  rpart      auc 0.8045154 0.05262589 5 0.02353501       0.95 0.7391717
6  rpart  logloss 0.6492840 0.15445732 5 0.06907541       0.95 0.4574999
7 ranger accuracy 0.7559322 0.04442057 5 0.01986548       0.95 0.7007768
8 ranger      auc 0.8299723 0.05334777 5 0.02385785       0.95 0.7637323
9 ranger  logloss 0.5055037 0.05966646 5 0.02668365       0.95 0.4314180
  conf_high tuned rank
1 0.8340411 FALSE    1
2 0.9065179 FALSE    1
3 0.5712943 FALSE    1
4 0.8206358 FALSE    3
5 0.8698591 FALSE    3
6 0.8410681 FALSE    3
7 0.8110876 FALSE    2
8 0.8962124 FALSE    2
9 0.5795894 FALSE    2
```

```
plot(cmp_cls)
```

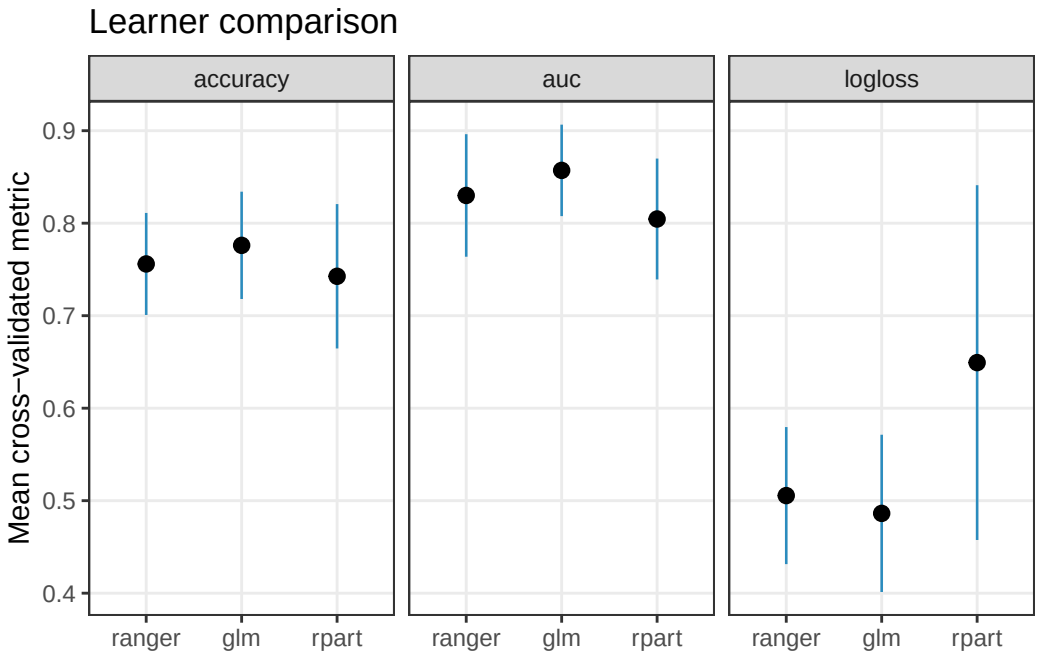

Figure 3: Comparison of GLM, CART, and random forest on the same folds.

8 Interpretation

8.1 Variable Importance

```
vip_obj <- interpret(
  fit_final,
  data_cls,
  method = "vip"
)

vip_obj$result
```

```
$scores
                  feature importance std_dev
1              chest_pain 0.12211221      NA
2              maximum_hr 0.10231023      NA
3             cholesterol 0.08910891      NA
4                     age 0.08250825      NA
5                     sex 0.08250825      NA
6                      bp 0.07920792      NA
7 exercise_induced_angina 0.07920792      NA
8             blood_sugar 0.00660066      NA

$baseline
```

```
[1] 0.9669967

$metric
[1] "accuracy"

$comparison
[1] "difference"

$raw_scores
NULL
```

```r
plot(vip_obj)
```

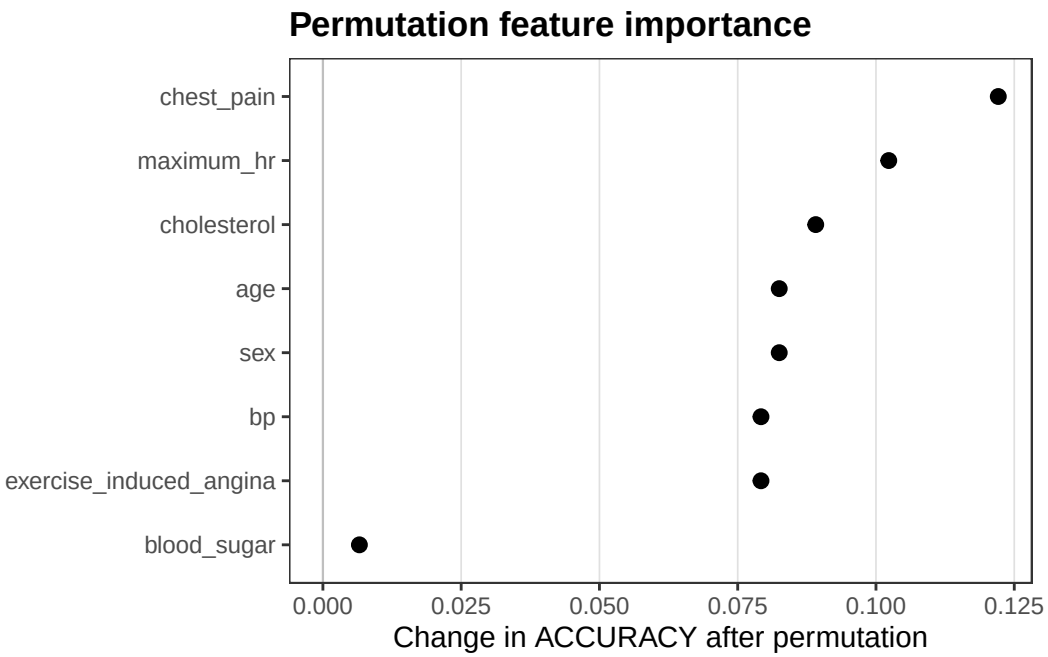

Figure 4: Model-based variable importance for the final random forest.

8.2 Permutation Importance

```r
perm_obj <- interpret(
  fit_final,
  data_cls,
  method = "permute",
  metric = "auc",
  type   = "prob",
  nsim   = 10,
  seed   = 42
)
```

```
perm_obj$result
```

```
$scores
                   feature  importance     std_dev
1              maximum_hr 0.059431479 0.007259506
2              chest_pain 0.056466047 0.005975871
3                     age 0.045683453 0.003159139
4                     sex 0.039844710 0.005478602
5 exercise_induced_angina 0.032795227 0.006491056
6             cholesterol 0.032067029 0.002837735
7                      bp 0.025728198 0.001839786
8             blood_sugar 0.002500439 0.001046502

$baseline
[1] 0.9929374

$metric
[1] "auc"

$comparison
[1] "difference"

$raw_scores
              age         sex chest_pain          bp cholesterol  blood_sugar
 [1,] 0.04763994 0.03491841 0.06790665 0.02816284  0.03605896 0.0036409896
 [2,] 0.04606071 0.03307598 0.05426391 0.02684682  0.03158449 0.0025443060
 [3,] 0.04241972 0.03684857 0.05575540 0.02548693  0.02978593 0.0017546938
 [4,] 0.04790314 0.04509563 0.06194069 0.02706615  0.03755045 0.0026320407
 [5,] 0.05079839 0.03689244 0.05930865 0.02575013  0.02785576 0.0013160204
 [6,] 0.04829795 0.03272504 0.05268468 0.02553079  0.03154062 0.0007457449
 [7,] 0.04562204 0.04233199 0.04763994 0.02702228  0.03167222 0.0040357958
 [8,] 0.04338480 0.04614845 0.05193894 0.02425864  0.03254957 0.0024565713
 [9,] 0.04013862 0.04579751 0.05246534 0.02561853  0.03053167 0.0023249693
[10,] 0.04456922 0.04461309 0.06075627 0.02153887  0.03154062 0.0035532550
      maximum_hr exercise_induced_angina
 [1,] 0.07374101              0.03474294
 [2,] 0.06123881              0.04255132
 [3,] 0.05202667              0.02996140
 [4,] 0.05619407              0.03645376
 [5,] 0.06615196              0.02188981
 [6,] 0.06404632              0.02408317
 [7,] 0.05382523              0.03540095
 [8,] 0.05084225              0.02934725
 [9,] 0.06172135              0.03939288
[10,] 0.05452711              0.03412879
```

```
plot(perm_obj)
```

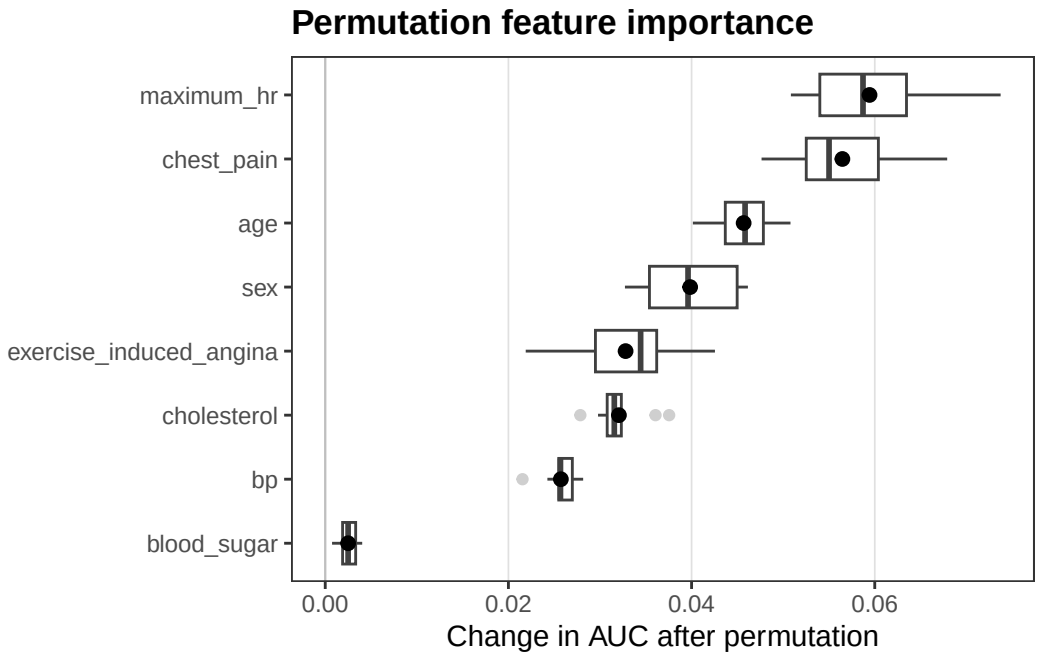

Figure 5: Permutation importance using AUC as the performance metric.

8.3 Calibration

```
cal_obj <- interpret(
  fit_final,
  data_cls,
  method    = "calibration",
  type      = "prob",
  pos_level = "Yes",
  bins      = 8
)

cal_obj$result
```

```
$curve
  bin     lower     upper   midpoint  n  mean_pred  observed    abs_gap
1   1 0.0000000 0.1058459 0.05292294 38 0.06251078 0.0000000 0.06251078
2   2 0.1058459 0.1569613 0.13140357 38 0.12970967 0.0000000 0.12970967
3   3 0.1569613 0.2547232 0.20584221 38 0.19943575 0.0000000 0.19943575
4   4 0.2547232 0.4136365 0.33417984 38 0.32077452 0.0000000 0.32077452
5   5 0.4136365 0.6103080 0.51197226 37 0.52222090 0.7567568 0.23453586
6   6 0.6103080 0.7576254 0.68396669 38 0.68536971 0.9210526 0.23568292
7   7 0.7576254 0.8913420 0.82448372 38 0.82265090 1.0000000 0.17734910
```

```
8   8 0.8913420 1.0000000 0.94567102 38 0.92797224 1.0000000 0.07202776

$prob
  [1] 0.27646614 0.92556703 0.89099666 0.06849058 0.02775201 0.16470734
  [7] 0.58183969 0.37658230 0.75142533 0.74169370 0.39048046 0.16902919
 [13] 0.71779376 0.07609073 0.14956520 0.22699075 0.43228179 0.34158149
 [19] 0.12882202 0.08638996 0.32134662 0.33062484 0.56682977 0.50583896
 [25] 0.92849230 0.05344600 0.14590739 0.26260793 0.26164212 0.85283795
 [31] 0.15612935 0.85410674 0.51172334 0.48821701 0.13877601 0.20528381
 [37] 0.89611856 0.96059615 0.93628752 0.49919457 0.61003949 0.26977213
 [43] 0.28216527 0.26425903 0.56319664 0.53579021 0.36261143 0.75803273
 [49] 0.18560867 0.13368002 0.05998578 0.48610531 0.64816309 0.04525802
 [55] 0.95132440 0.91950779 0.42161334 0.62115534 0.17876712 0.41363652
 [61] 0.78759244 0.15558680 0.90601537 0.07501097 0.73038710 0.97740141
 [67] 0.54270304 0.26421795 0.94716292 0.52526311 0.18630196 0.70279464
 [73] 0.90661471 0.75721804 0.44223695 0.16114090 0.96337277 0.12679097
 [79] 0.07810939 0.94058538 0.31296495 0.22173429 0.13859114 0.73441526
 [85] 0.10700677 0.11423961 0.17844877 0.11309070 0.10470903 0.05697605
 [91] 0.41014599 0.66260827 0.49688449 0.07885097 0.17613626 0.74516198
 [97] 0.87207060 0.67955227 0.13219929 0.21608973 0.19532134 0.18857996
[103] 0.33089660 0.15225052 0.56728413 0.22225987 0.72069973 0.55694949
[109] 0.95306483 0.65020897 0.91001100 0.94340806 0.19024952 0.81772861
[115] 0.49167168 0.19701937 0.25216317 0.11219528 0.93859273 0.82641752
[121] 0.82499920 0.62903909 0.23056473 0.89859016 0.60065685 0.07279706
[127] 0.92688276 0.86753300 0.06379128 0.23295849 0.31342238 0.15779319
[133] 0.07560127 0.41068617 0.03650772 0.10032451 0.88634396 0.77844241
[139] 0.89561643 0.12544024 0.28292579 0.67175457 0.08515752 0.76980056
[145] 0.28739953 0.43897774 0.84465367 0.11398760 0.11586281 0.10877623
[151] 0.28543794 0.15122456 0.10815541 0.94161215 0.85357694 0.77144938
[157] 0.65181196 0.71513538 0.75829916 0.29484822 0.14229987 0.78847499
[163] 0.05164740 0.20710412 0.07699154 0.39974441 0.16553409 0.25408585
[169] 0.82214960 0.13286122 0.78175165 0.62347239 0.92744796 0.47375959
[175] 0.74403335 0.91960906 0.33082552 0.89328963 0.11832265 0.07542392
[181] 0.59737859 0.74719516 0.18299695 0.42632384 0.68014152 0.21732686
[187] 0.06258889 0.69306402 0.58089448 0.62456373 0.17656481 0.94163122
[193] 0.89540646 0.73531341 0.12580977 0.80466630 0.32605212 0.25109258
[199] 0.06185104 0.71862705 0.14781745 0.53688369 0.18039128 0.18307075
[205] 0.37285195 0.89762429 0.94937890 0.83505373 0.15431934 0.76727503
[211] 0.02563272 0.53742867 0.10402078 0.76700250 0.64174097 0.30798573
[217] 0.04417626 0.25887447 0.38698149 0.37728984 0.12070925 0.05013353
[223] 0.04383702 0.90646307 0.64627221 0.03313093 0.45006766 0.14213991
[229] 0.83307516 0.82813834 0.03481331 0.87611354 0.61039749 0.35730565
[235] 0.11166678 0.96320337 0.93426331 0.72175810 0.04010899 0.13613685
[241] 0.26107781 0.09016972 0.13143412 0.56495193 0.23520034 0.81182903
[247] 0.61794984 0.92319681 0.53770544 0.25663505 0.66283610 0.83447110
[253] 0.66805935 0.04664537 0.19134522 0.02504939 0.20237531 0.16139319
[259] 0.34368230 0.64219429 0.06818702 0.60979982 0.21137353 0.12335664
[265] 0.94034194 0.95609300 0.79829688 0.64018236 0.51707020 0.21935549
```

```
[271] 0.91104833 0.45640385 0.93967289 0.23957167 0.57058657 0.31601952
[277] 0.11753837 0.05069545 0.55078309 0.27660471 0.89237820 0.09142372
[283] 0.81989136 0.05363394 0.70323537 0.81569663 0.90507214 0.27077943
[289] 0.12842453 0.14129961 0.59121468 0.16262816 0.87151060 0.87431776
[295] 0.81777854 0.10622482 0.77066507 0.85052850 0.71211385 0.67988001
[301] 0.87716634 0.45562344 0.12032841

$truth
  1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
 No Yes Yes  No  No  No Yes  No Yes Yes  No  No Yes  No  No  No Yes  No  No  No
 21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40
 No  No Yes Yes Yes  No  No  No  No Yes  No Yes Yes  No  No  No Yes Yes Yes  No
 41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60
Yes  No  No  No Yes Yes  No Yes  No  No  No  No Yes  No Yes Yes Yes Yes  No  No
 61  62  63  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78  79  80
Yes  No Yes  No Yes Yes Yes  No Yes Yes  No Yes Yes Yes Yes  No Yes  No  No Yes
 81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96  97  98  99 100
 No  No  No Yes  No  No  No  No  No  No  No Yes  No  No  No Yes Yes Yes  No  No
101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120
 No  No  No  No Yes  No Yes Yes Yes Yes Yes Yes  No Yes Yes  No  No  No Yes Yes
121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140
Yes Yes  No Yes Yes  No Yes Yes  No  No  No  No  No  No  No  No Yes Yes Yes  No
141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160
 No Yes  No Yes  No Yes Yes  No  No  No  No  No  No Yes Yes Yes Yes Yes Yes  No
161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
 No Yes  No  No  No  No  No  No Yes  No Yes  No Yes  No Yes Yes  No Yes  No  No
181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200
Yes Yes  No  No Yes  No  No Yes Yes Yes  No Yes Yes Yes  No Yes  No  No  No Yes
201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220
 No  No  No  No  No Yes Yes Yes  No Yes  No Yes  No Yes Yes  No  No  No  No  No
221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240
 No  No  No Yes Yes  No  No  No Yes Yes  No Yes Yes  No  No Yes Yes Yes  No  No
241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260
 No  No  No Yes  No Yes Yes Yes Yes  No  No Yes  No  No  No  No  No  No  No Yes
261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280
 No Yes  No  No Yes Yes Yes Yes Yes  No Yes  No Yes  No Yes  No  No  No Yes  No
281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300
Yes  No Yes  No Yes Yes Yes  No  No  No Yes  No Yes Yes Yes  No Yes Yes Yes Yes
301 302 303
Yes Yes  No
Levels: No Yes

$positive
[1] "Yes"

$ece
[1] 0.17882
```

```
$mce
[1] 0.3207745
```

```r
plot(cal_obj, style = "curve")
```

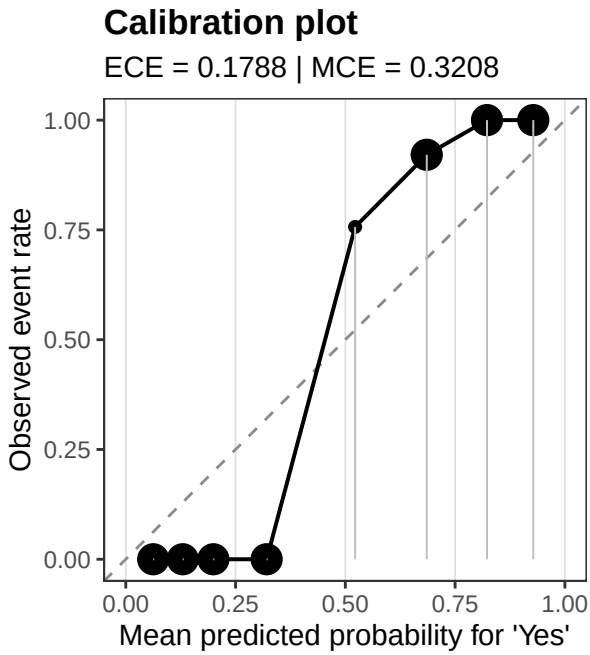

Figure 6: Calibration curve for predicted heart disease probability.

9 Optional SHAP Code

SHAP can be informative, but it is intentionally not evaluated here because it is computationally expensive during document rendering.

```r
high_risk <- data_cls[which.max(final_prob[, "Yes"]), , drop = FALSE]

shap_hr <- interpret(
  fit_final,
  data_cls,
  method  = "shap",
  newdata = high_risk,
  type    = "prob",
  nsim    = 50,
  seed    = 42
)

plot(shap_hr, kind = "waterfall")
```

10 Export Results

The following chunk writes the dataset, formula, fitted objects, tabular outputs, predictions, plots, and session information to `heartdisease_pipeline_results/`.

```
write_export <- function(x, path) {
  tab <- tryCatch(
    {
      if (!is.data.frame(x)) {
        x <- as.data.frame(x)
      }
      x[] <- lapply(x, function(col) {
        if (is.list(col)) {
          vapply(col, function(value) {
            paste(capture.output(str(value, max.level = 1)), collapse = " ")
          }, character(1))
        } else {
          col
        }
      })
      x
    },
    error = function(e) {
      data.frame(output = capture.output(print(x)))
    }
  )

  write.csv(tab, path, row.names = FALSE)
}

write.csv(data_cls, file.path(results_dir, "heartdisease_classification_data.csv"),
          row.names = FALSE)
writeLines(deparse(formula_cls), file.path(results_dir, "model_formula.txt"))

write.csv(predictions, file.path(results_dir, "baseline_predictions.csv"),
          row.names = FALSE)
write.csv(final_predictions, file.path(results_dir, "final_predictions.csv"),
          row.names = FALSE)

write_export(eval_baseline$summary,
             file.path(results_dir, "baseline_evaluation_summary.csv"))
write_export(tuned_ranger$results,
             file.path(results_dir, "tuning_results.csv"))
write_export(tuned_ranger$best,
             file.path(results_dir, "best_hyperparameters.csv"))
write_export(cmp_cls$results,
             file.path(results_dir, "learner_comparison.csv"))
```

```
write_export(vip_obj$result,
             file.path(results_dir, "variable_importance.csv"))
write_export(perm_obj$result,
             file.path(results_dir, "permutation_importance.csv"))
write_export(cal_obj$result,
             file.path(results_dir, "calibration_table.csv"))

saveRDS(fit_baseline, file.path(results_dir, "baseline_fit.rds"))
saveRDS(eval_baseline, file.path(results_dir, "baseline_evaluation.rds"))
saveRDS(tuned_ranger, file.path(results_dir, "tuned_ranger.rds"))
saveRDS(fit_final, file.path(results_dir, "final_fit.rds"))
saveRDS(cmp_cls, file.path(results_dir, "learner_comparison.rds"))
saveRDS(vip_obj, file.path(results_dir, "variable_importance.rds"))
saveRDS(perm_obj, file.path(results_dir, "permutation_importance.rds"))
saveRDS(cal_obj, file.path(results_dir, "calibration.rds"))

pdf(file.path(results_dir, "baseline_evaluation_plot.pdf"), width = 7, height = 5)
print(plot(eval_baseline))
dev.off()
```

```
pdf
  2
```

```
pdf(file.path(results_dir, "tuning_plot.pdf"), width = 7, height = 5)
print(plot(tuned_ranger))
dev.off()
```

```
pdf
  2
```

```
pdf(file.path(results_dir, "learner_comparison_plot.pdf"), width = 7, height = 5)
print(plot(cmp_cls))
dev.off()
```

```
pdf
  2
```

```
pdf(file.path(results_dir, "variable_importance_plot.pdf"), width = 7, height = 5)
print(plot(vip_obj))
dev.off()
```

```
pdf
  2
```

```
pdf(file.path(results_dir, "permutation_importance_plot.pdf"), width = 7, height = 5)
print(plot(perm_obj))
dev.off()
```

```
pdf
  2
```

```
pdf(file.path(results_dir, "calibration_plot.pdf"), width = 7, height = 5)
print(plot(cal_obj, style = "curve"))
dev.off()
```

```
pdf
  2
```

```
writeLines(capture.output(sessionInfo()),
           file.path(results_dir, "session_info.txt"))

sort(list.files(results_dir))
```

```
 [1] "baseline_evaluation_plot.pdf"
 [2] "baseline_evaluation_summary.csv"
 [3] "baseline_evaluation.rds"
 [4] "baseline_fit.rds"
 [5] "baseline_predictions.csv"
 [6] "best_hyperparameters.csv"
 [7] "calibration_plot.pdf"
 [8] "calibration_table.csv"
 [9] "calibration.rds"
[10] "final_fit.rds"
[11] "final_predictions.csv"
[12] "heartdisease_classification_data.csv"
[13] "learner_comparison_plot.pdf"
[14] "learner_comparison.csv"
[15] "learner_comparison.rds"
[16] "model_formula.txt"
[17] "permutation_importance_plot.pdf"
[18] "permutation_importance.csv"
[19] "permutation_importance.rds"
[20] "session_info.txt"
[21] "tuned_ranger.rds"
[22] "tuning_plot.pdf"
[23] "tuning_results.csv"
[24] "variable_importance_plot.pdf"
[25] "variable_importance.csv"
[26] "variable_importance.rds"
```

11 Reuse

The exported `.rds` files preserve the fitted **funcml** objects for later R analysis. The `.csv` files are plain tabular outputs for reporting, auditing, or sharing outside R. The plot PDFs can be inserted directly into reports or presentations.